

Introduction to Threads

What is a Thread?

Within a program, a thread is a separate execution path. It is a lightweight process that the operating system can schedule and run concurrently with other threads. The operating system creates and manages threads, and they share the same memory and resources as the program that created them. This enables multiple threads to collaborate and work efficiently within a single program.

A thread is a single sequence stream within a process. Threads are also called as **lightweight processes** as it possess some of the properties of processes. Each thread belongs to exactly one process.

In operating system that support multithreading, process can consist of many threads.

These threads run in parallel improving the application performance.

Each such thread has its own CPU state and stack, but they share the address space of the process and the environment.

Threads can share common data so they do not need to use inter-process communication.

Like the processes, threads also have states like ready, executing, blocked etc. priority can be assigned to the threads just like process and highest priority thread is scheduled first.

Each thread has its own Thread Control Block (TCB). Like process, context switch occurs for the thread and register contents are saved in (TCB).

As threads share the same address space and resources, synchronization is also required for the various activities of the thread.

Why Multithreading?

A thread is also known as lightweight process. The idea is to achieve parallelism by dividing a process into multiple threads. For example, in MS Word uses multiple threads: one thread to format the text, another thread to process inputs, etc.

Multithreading is a technique used in operating systems to improve the performance and responsiveness of computer systems. Multithreading allows multiple threads (i.e., lightweight processes) to share the same resources of a single process, such as the CPU, memory, and I/O devices.

Process vs Thread:

S.NO	Process	Thread
1.	Process means any program is in execution.	Thread means a segment of a process.
2.	The process takes more time to terminate.	The thread takes less time to terminate.
3.	It takes more time for creation.	It takes less time for creation.
4.	It also takes more time for context switching.	It takes less time for context switching.
5.	The process is less efficient in terms of communication.	Thread is more efficient in terms of communication.
6.	Multiprogramming holds the concepts of multi-process.	We don't need multi programs in action for multiple threads because a single process consists of multiple threads.
7.	The process is isolated.	Threads share memory.
8.	The process is called the heavyweight process.	A Thread is lightweight as each thread in a process shares code, data, and resources.
9.	Process switching uses an interface in an operating system.	Thread switching does not require calling an operating system and causes an interrupt to the kernel.
10.	If one process is blocked then it will not affect the execution of other processes	If a user-level thread is blocked, then all other user-level threads are blocked.
11.	The process has its own Process Control Block, Stack, and Address Space.	Thread has Parents' PCB, its own Thread Control Block, and Stack and common Address space.
12.	Changes to the parent process do not affect child processes.	Since all threads of the same process share address space and other resources so any changes to the main thread may affect the behavior of the other threads of the process.
13.	A system call is involved in it.	No system call is involved, it is created using APIs.
14.	The process does not share data with each other.	Threads share data with each other.

Advantages of Thread over Process

1. **Responsiveness:** If the process is divided into multiple threads, if one thread completes its execution, then its output can be immediately returned.
2. **Faster context switch:** Context switch time between threads is lower compared to process context switch. Process context switching requires more overhead from the CPU.

3. **Effective utilization of multiprocessor system:** If we have multiple threads in a single process, then we can schedule multiple threads on multiple processor. This will make process execution faster.
4. **Resource sharing:** Resources like code, data, and files can be shared among all threads within a process. Note: stack and registers can't be shared among the threads. Each thread has its own stack and registers.
5. **Communication:** Communication between multiple threads is easier, as the threads shares common address space. while in process we have to follow some specific communication technique for communication between two process.
6. **Enhanced throughput of the system:** If a process is divided into multiple threads, and each thread function is considered as one job, then the number of jobs completed per unit of time is increased, thus increasing the throughput of the system.

Types of Threads

Types of Threads:

1. **User Level thread (ULT)** – Is implemented in the user level library, they are not created using the system calls. Thread switching does not need to call OS and to cause interrupt to Kernel. Kernel doesn't know about the user level thread and manages them as if they were single-threaded processes.
 - **Advantages of ULT** –
 - Can be implemented on an OS that doesn't support multithreading.
 - Simple representation since thread has only program counter, register set, stack space.
 - Simple to create since no intervention of kernel.
 - Thread switching is fast since no OS calls need to be made.
 - **Limitations of ULT** –
 - No or less co-ordination among the threads and Kernel.
 - If one thread causes a page fault, the entire process blocks.
2. **Kernel Level Thread (KLT)** – Kernel knows and manages the threads. Instead of thread table in each process, the kernel itself has thread table (a master one) that keeps track of all the threads in the system. In addition kernel also maintains the traditional process table to keep track of the processes. OS kernel provides system call to create and manage threads.
 - **Advantages of KLT** –
 - Since kernel has full knowledge about the threads in the system, scheduler may decide to give more time to processes having large number of threads.
 - Good for applications that frequently block.
 - **Limitations of KLT** –
 - Slow and inefficient.
 - It requires thread control block so it is an overhead.

Difference between User-Level Thread V/S Kernel-Level Thread

S. No.	Parameters	User Level Thread	Kernel Level Thread
1.	Implemented by	User threads are implemented by users.	Kernel threads are implemented by Operating System (OS).
2.	Recognize	The operating System doesn't recognize user-level threads.	Kernel threads are recognized by Operating System.
3.	Implementation	Implementation of User threads is easy.	Implementation of Kernel-Level thread is complicated.
4.	Context switch time	Context switch time is less.	Context switch time is more.
5.	Hardware support	Context switch requires no hardware support.	Hardware support is needed.
6.	Blocking operation	If one user-level thread performs a blocking operation then the entire process will be blocked.	If one kernel thread performs a blocking operation then another thread can continue execution.
7.	Multithreading	Multithread applications cannot take advantage of multiprocessing.	Kernels can be multithreaded.
8.	Creation and Management	User-level threads can be created and managed more quickly.	Kernel-level level threads take more time to create and manage.
9.	Operating System	Any operating system can support user-level threads.	Kernel-level threads are operating system-specific.
10.	Thread Management	The thread library contains the code for thread creation, message passing, thread scheduling, data transfer, and thread destroying	The application code does not contain thread management code. It is merely an API to the kernel mode. The Windows operating system makes use of this feature.
11.	Example	Example: Java thread , POSIX threads.	Example: Window Solaris.
12.	Advantages	• User Level Threads are simple and quick to create. • Can run on any operating system • They perform better than kernel threads since they don't need to make system calls to create threads. • In user-level threads, switching between	• Scheduling multiple threads that belong to the same process on different processors is possible in kernel-level threads. • Multithreading can be there for kernel routines. • When a thread at the kernel level is halted, the kernel can schedule another thread for the same process.

S. No.	Parameters	User Level Thread	Kernel Level Thread
		threads does not need kernel mode privileges.	
13.	Disadvantages	• Multithreaded applications on user-level threads cannot benefit from multiprocessing. • If a single user-level thread performs a blocking operation, the entire process is halted.	• Transferring control within a process from one thread to another necessitates a mode switch to kernel mode. • Kernel-level threads take more time to create and manage than user-level threads.